

RTL Design Sherpa

Reed-Solomon Board Validation Report Genesys 2 FPGA board evidence

October 2026

Table of Contents

1 Document Information....8
1.1 Revision History....8
1.2 Scope....9
1.3 Provenance....10
2 Scope and Claims....11
2.1 What this book is....11
2.2 What is claimed....12
2.3 What is not claimed....13
3 Images and Codec Profile....14
3.1 Image matrix....14
3.2 Codec profile....15
4 Harness Architecture....16
4.1 RTL inventory....16
4.2 Host and build inventory....18
4.3 Datapath diagram....19
4.3.1 Figure 2.1: Reed-Solomon AXIS datapath and register path....19
4.4 Register path, clock, and reset....20
5 Campaigns and the Oracle Argument....21
5.1 Campaign inventory....21
5.2 Determinism and cross-solver agreement....22
5.3 The dual-solver loop as oracle....23
6 Testing Choices....24

6.1 Injector modes mapped to failure mechanisms....24
6.2 Post-encoder injection protects parity symbols....25
6.3 UART and the safety pair....26
6.4 Genesys 2 vs Nexys A7....27
6.5 AXIS and AXI4 flavors cover both integration styles....28
6.6 Deliberately not board-tested....29
7 Erasure Campaign....30
7.1 What erasure decoding adds....30
7.2 Board findings....31
7.3 TASK-005 boundary....32
8 Battery Results....33
8.1 Per-image campaign status....33
8.2 Correction boundary....34
8.2.1 Figure 4.1: Reed-Solomon correction boundary across all four images. 34
8.3 Decode cost....35
8.3.1 Figure 4.2: Reed-Solomon decode cost across all four images....35
8.4 Sweep verdict table....36
8.5 Solver agreement....38
8.5.1 Figure 4.3: Reed-Solomon solver agreement matrix (riBM vs Euclid)....38
9 Soak Results....39
9.1 One million blocks on the AXI4 image....39
9.2 Mis-decodes versus the model....40
9.3 Soak timeline....41

9.3.1 Figure 4.4: RS soak timeline over one million blocks....41
9.4 What 29 blk/s means....42
9.5 Image notes....43
10 Honest Limits....44
10.1 What the board evidence does not cover....44
10.2 Traceability....45

List of Figures

Figure 2.1: Reed-Solomon AXIS datapath and register path.....19
Figure 4.1: Reed-Solomon correction boundary across all four images.....34
Figure 4.2: Reed-Solomon decode cost across all four images.....35
Figure 4.3: Reed-Solomon solver agreement matrix (riBM vs Euclid).....38
Figure 4.4: RS soak timeline over one million blocks.....41

List of Tables

Table 1: Table 0.1: Revision history....8
Table 2: Table 1.1: Reed-Solomon Genesys 2 image matrix....14
Table 3: Table 2.1: RTL blocks in the Reed-Solomon harness....16
Table 4: Table 2.2: Host scripts and build flow....18
Table 5: Table 3.1: Reed-Solomon board campaign inventory....21
Table 6: Table 3.2: Reed-Solomon erasure boundary results....31
Table 7: Table 4.1: Reed-Solomon 2026-10-05 battery per-image status....33
Table 8: Table 4.2: Reed-Solomon sweep verdicts....36
Table 9: Table 4.1: RS soak final counters....39
Table 10: Table 5.1: Traceability from claim to artifact....45

1 Document Information

1.1 Revision History

Table 0.1: Revision history.

Revision	Date	Author	Description
0.1	2026-10-05	RTL Design Sherpa	Initial board validation roll-up for the Reed-Solomon Genesys 2 harness.
0.2	2026-10-20	RTL Design Sherpa	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
0.3	2026-10-20	RTL Design Sherpa	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

1.2 Scope

This book covers the RS(252,236) t=8 codec as exercised on the Digilent Genesys 2 board. It describes the harness, the campaign methodology, the 2026-10-05 battery, the erasure boundary finding, and the completed million-block soak.

1.3 Provenance

All timing and campaign results in this book come from `stable/MANIFEST.md`, `stable/results/2026-10-05_battery/`, and `stable/results/2026-10-05_soak/`. The board used for the runs is Digilent Genesys 2 serial `200300B818A0` (Kintex-7 `xc7k325t-2`), programmed and measured with Vivado 2025.1.

2 Scope and Claims

2.1 What this book is

This report is the board's statement about the RS(252,236) t=8 codec. The Genesys 2 harness runs the same campaigns in cocotb simulation and on real silicon, so when the board passes we have evidence that simulation matches hardware at 100 MHz with real I/O, real clocks, and real register traffic.

2.2 What is claimed

The 2026-10-05 battery demonstrates that four single-decoder bitstreams (two flavours times two solvers) pass the same deterministic campaign set:

- `init`, `smoke`, `sweep`, `random`, `clusters`, `localized`, and `badblock` all PASS on `axis_ribm`, `axis_euclid`, `axi4_ribm`, and `axi4_euclid`.
- The sweep shows e <= 8 corrected, e = 9 .. 18 uncorrectable, on every image.
- The random campaign recorded 64 runs x 16 blocks with 0 failing runs across all images.
- The matching verdict counts between riBM and Euclid images are themselves a cross-solver check.
- The 2026-10-05 million-block soak on `axi4_ribm` passed: 0 failing runs, and 2 mis-decodes out of 74,240 over-t blocks, below the model's approximately 1-in-20,000 bound at e = t + 1.

That is board evidence that both key-equation solvers behave identically under the same workload and that the codec honours the correction boundary.

2.3 What is not claimed

This book does not replace the component DV matrix. It does not prove every possible error pattern or every smaller RS profile. The dual-decoder on-chip comparator is present in simulation but not in the production board matrix; agreement is proven by running identical deterministic campaigns on the two single-decoder images. The erasure `f = 2t` boundary is a known open issue (TASK-005) and is called out in Chapter 3 and Chapter 5. The million-block soak completed on 2026-10-06; Chapter 4 reports its counters.

3 Images and Codec Profile

3.1 Image matrix

Four Genesys 2 bitstreams were built and kept in `stable/reports/`:

Table 1.1: Reed-Solomon Genesys 2 image matrix.

Image	Solver	Bitstream	WNS (ns)	Failing endpoints	Notes
axis_ribm	riBM	`rs_loop_genesys2_axis_ribm.bit`	+1.072	0	Streaming pipe.
axis_euclid	Euclid	`rs_loop_genesys2_axis_euclid.bit`	+0.875	0	Streaming pipe.
axi4_ribm	riBM	`rs_loop_genesys2_axi4_ribm.bit`	+1.822	0	Memory-to-memory chain.
axi4_euclid	Euclid	`rs_loop_genesys2_axi4_euclid.bit`	+1.431	0	Memory-to-memory chain.

All four images are timing-clean at 100 MHz on the `k325t-2`. The WNS values were read from the four `stable/reports/genesys2_*/timing_summary.txt` files.

3.2 Codec profile

The geometry is frozen in `build-loop/rtl/rs_loop_cfg_pkg.sv`:

- Code: RS(252,236), t=8, m=8.
- Symbols per beat: 4 (S=4), so a 32-bit beat carries four 8-bit symbols.
- Shortened from the natural 255-symbol RS block so that n=252 and k=236 are both multiples of the beat size. A full codeword is 63 beats; the message is 59 beats.

The shortening was deliberate: it makes the AXIS datapath line-rate at 63 cycles per block and keeps the AXI4 memory alignment clean. The host reads `PROFILE` to discover the geometry and `TOPOLOGY` to discover which datapath and solver are loaded.

4 Harness Architecture

4.1 RTL inventory

Table 2.1: RTL blocks in the Reed-Solomon harness.

Module	File	Role
`uart_axil_bridge`	`projects/components/utility-ip/converters/rtl/uart_to_axil4/uart_axil_bridge.sv`	UART 115200 8N1 to AXI4-Lite master.
`bridge_rs_loop_axil`	Generated fabric	1-master x 3-slave AXI4-Lite/APB fabric.
`rs_loop_genesys2_top` / `rs_loop_top`	`build-loop/rtl/rs_loop_genesys2_top.sv` / `build-loop/rtl/rs_loop_top.sv`	Genesys 2 MMCM wrapper, or Nexys A7 100 MHz direct clock.
`rs_loop_harness`	`build-loop/rtl/rs_loop_harness.sv`	Codec loop, register decode, bandwidth meters, observers, verdict tallies.
`rs_loop_cfg_pkg`	`build-loop/rtl/rs_loop_cfg_pkg.sv`	Single source of geometry: RS(252,236), 4 symbols/beat.
`rs_loop_regs`	Generated from `build-loop/rtl/rs_loop_regs.rdl`	Host-visible CSR block.
`axis4_master_injector`	`rtl/amba/shared/axis4_master_injector.sv`	LFSR data source plus expected CRC-32, one packet per block.
`axis4_slave_pattern_check`	`rtl/amba/shared/axis4_slave_pattern_check.sv`	Regenerates the same LFSR pattern and compares beats per decoder.
`rs_encoder_core`	Component RS RTL	RS(252,236) encoder.
`error_injector`	Shared utility RTL	Post-encoder corruption:

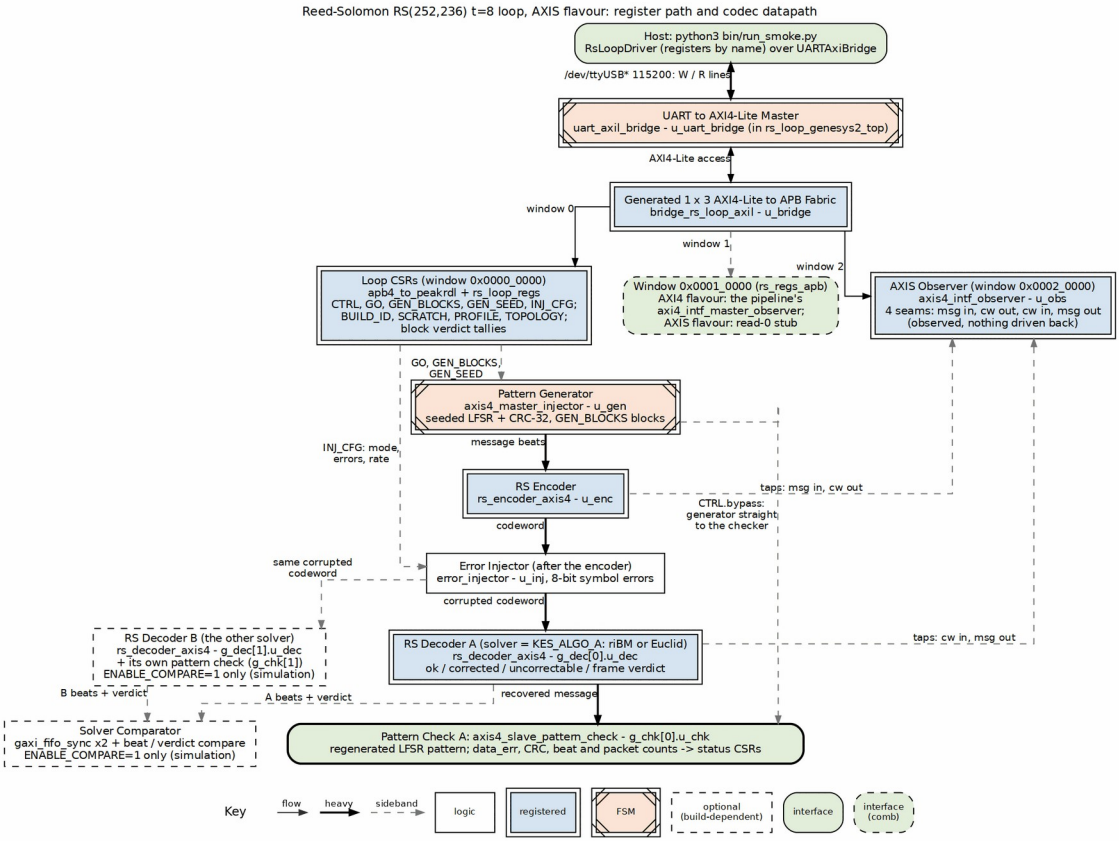
Module	File	Role
		modes 0..7 plus erasure mark.
rs_decoder_core	Component RS RTL	Decoder with selectable KES_ALGO and erasure path.
rs_erasure_unit	Component RS RTL	Erasure locator / transform used when ERASURE_SUPPORT=1.
rs_axi4_pipeline	build-loop/rtl/rs_axi4_pipeline.sv	Memory-to-memory job chain for the AXI4 flavor.
axis4_intf_observer	Shared utility RTL	Four AXIS seam observer on window 0x20000.
axi4_intf_master_observer	Shared utility RTL	Four codec AXI4 master-port observer on window 0x10000.

4.2 Host and build inventory

Table 2.2: Host scripts and build flow.

Script / target	File	Role
`rs_env.py`	`bin/rs_env.py`	Path anchor for shared host code.
`run_smoke.py`	`bin/run_smoke.py`	Campaign runner.
`init` / `smoke` / `sweep` / `random` / `soak` / `erasure`	`bin/seq_*.py`	Deterministic campaign sequences.
`RsLoopDriver`	`build-loop/host/rs_loop.py`	By-name register access and status collection.
`rs_loop_programs`	`build-loop/host/rs_loop_programs.py`	Shared authored-once programs.
`host_rs_loop.py`	`build-loop/host/host_rs_loop.py`	CLI front-end.
`make -C build-loop bitstream`	`build-loop/Makefile`	Vivado build; set `RS_TARGET=genesys2`.
`bin/build_image_matrix.sh`	`bin/build_image_matrix.sh`	Builds `{axis,axi4}` x `{riBM,Euclid}`.

4.3 Datapath diagram



Reed-Solomon AXIS datapath and register path

4.3.1 Figure 2.1: Reed-Solomon AXIS datapath and register path.

Source: 01_rs_axis_loop.dot

4.4 Register path, clock, and reset

The host register path is Python over pyserial to the UART, into `uart_axil_bridge`, through the generated `bridge_rs_loop_axil` 1x3 fabric, and out as three APB windows. Window 0 at `0x00000000` holds `rs_loop_regs` (BUILD_ID, SCRATCH, PROFILE, TOPOLOGY, INJ_CFG, etc.). Window 1 at `0x00010000` hosts the AXI4 master observer on AXI4 flavours and is a read-zero stub on AXIS. Window 2 at `0x00020000` is always live and hosts the AXIS seam observer.

On the Genesys 2 the 200 MHz LVDS system clock passes through `IBUFDS` into an `MMCME2_BASE` with VCO = 1200 MHz and `CLKOUT0_DIVIDE_F = 12`, producing the 100 MHz harness clock. The Nexys A7 top uses the on-board 100 MHz clock directly.

5 Campaigns and the Oracle Argument

5.1 Campaign inventory

Table 3.1: Reed-Solomon board campaign inventory.

Campaign	What it proves	Determinism
`init`	BUILD_ID, SCRATCH round-trip, PROFILE, TOPOLOGY.	Deterministic.
`smoke`	Bypass, clean, e=t, e=t+1, deterministic DEBUG walk.	Deterministic.
`sweep`	Exact correction counts for e = 0 .. 2t+2.	Deterministic.
`random`	Fresh seeds, mixed injector modes, 64 runs x 16 blocks.	Host-seeded; reproducible on demand.
`clusters`	Spatially correlated errors.	Deterministic combo matrix.
`localized`	Windowed errors.	Deterministic combo matrix.
`badblock`	Retention-tail / worn-block behaviour.	Deterministic combo matrix.
`erasure`	Marked-symbol runs at f = t, 2t, 2t+1.	Deterministic.
`soak`	Long-running mixed-mode stress; the million-block run.	Host-seeded; replayable.

5.2 Determinism and cross-solver agreement

The host-side RNG seed defaults to 1. With that default, the random, soak, clusters, localized, and badblock campaigns are fully deterministic. Because the riBM and Euclid images run bit-identical workloads, the matching verdict counts between them are the cross-check. You do not need the on-chip comparator to know the two solvers agree; the campaigns prove it.

5.3 The dual-solver loop as oracle

The AXIS datapath in simulation feeds the same corrupted blocks to two `rs_decoder_core` instances, one with `KES_ALGO=RIBM` and one with `KES_ALGO=EUCLID`. Each decoder's output is compared beat-by-beat against the generator's regenerated LFSR pattern in its own `axis4_slave_pattern_check`, so a correction is judged by a reference that never saw the errors. A separate comparator then requires the two decoders to agree on every beat and on every block's verdict. On the board images this comparator is absent by construction; the cross-check is the deterministic campaign battery.

6 Testing Choices

6.1 Injector modes mapped to failure mechanisms

The shared `error_injector` operates on 8-bit symbols. Mode 1 exact-count places exactly e errors per block at uniformly random distinct positions, which is what makes the e = t / e = t + 1 boundary sharp. Mode 2 burst and mode 3 rate model raw channel bit-error behaviour. Modes 4 clusters and 5 localized model spatially correlated faults. Mode 6 badblock models the tail of NAND retention. Mode 7 debug is the deterministic bring-up walk.

6.2 Post-encoder injection protects parity symbols

Errors are injected after the encoder, not in the generator. An error in the generator's data would be encoded faithfully and would be invisible to the code. Placing the injector on the coded stream means parity symbols are corrupted too, which is the workload the decoder actually has to handle.

6.3 UART and the safety pair

The harness uses a plain UART instead of a proprietary debug core, so the host side is just pyserial and Python scripts. Board safety is a pair: `board_lock.sh` serializes programming flows by JTAG serial, and `fpga_board.py` reads the JTAG chain before programming to confirm the expected Genesys 2 serial (`200300B818A0`) is present with a device behind it.

6.4 Genesys 2 vs Nexys A7

The Kintex-7 XC7K325T-2 was chosen because it has room for the full harness plus observers and still closes timing with positive slack. The Nexys A7-100T flow is kept as a target option (`RS_TARGET=nexys_a7_100t`) but the Genesys 2 is the primary target for harness-class work.

6.5 AXIS and AXI4 flavors cover both integration styles

The AXIS flavor is a single streaming pipe; the AXI4 flavor replaces the middle with `rs_axi4_pipeline`, a memory-to-memory job chain over block RAM. Both use the same generator, checker, CSRs, and verdict logic, so a run reports itself identically whichever middle is built. Running the same deterministic campaign on both flavours is a free sanity check. The AXI4 flavor refuses a `GEN_BLOCKS` value that would exceed the job memories.

6.6 Deliberately not board-tested

The dual-decoder comparator with `ENABLE_COMPARE=1` is the sim/DV domain. The soak campaign runs separately from the battery because of its wall time — hours at UART rates; it completed on 2026-10-06 and is reported in Chapter 4. The component DV matrix exercises the codec in ways the board harness does not replicate. The erasure `f = 2t` boundary is a known open issue, detailed in the next section.

7 Erasure Campaign

7.1 What erasure decoding adds

When `INJ_CFG.mark` is set, the injector's hit mask rides the decoder `in_erasure` sideband. The decoder is then told which symbols were corrupted, so the correction bound doubles: the decoder can correct e errors plus f erasures whenever 2e + f <= 2t. For RS(252,236) t=8, that means:

- f = t = 8 erasures alone should correct.
- f = 2t = 16 erasures alone should correct.
- f = 2t + 1 = 17 erasures should be refused by inspection.

7.2 Board findings

The `erasure` sequence was run on the Genesys 2 board:

Table 3.2: Reed-Solomon erasure boundary results.

f	Expected	Observed
8 (t)	Correct	Correct: 16/16 blocks corrected, A=B solver agreement.
16 (2t)	Correct	Flagged uncorrectable: `ok/corr/unc = 0/0/16`.
17 (2t+1)	Refuse by inspection	Refused: 16/16 blocks uncorrectable.

7.3 TASK-005 boundary

The f = 2t case is the known open issue TASK-005 (`vault/tracking/projects/components/ecc-ip/reed-solomon/task/open/TASK-005.md`): the B-stage `t_zero` bypass intends to pass f = 16, but the final verdict fires downstream. The f = t and f = 2t+1 paths are correct; the f = 2t path must be fixed in the component DV and then re-run on the board. This report documents the finding exactly as measured and does not fabricate a corrected result.

8 Battery Results

8.1 Per-image campaign status

All four images passed every campaign in the 2026-10-05 battery:

Table 4.1: Reed-Solomon 2026-10-05 battery per-image status.

Image	Solver	Init	Smoke	Sweep	Random	Clusters	Localized	Badblock
axis_ribm	riBM	PASS	PASS	PASS	PASS	PASS	PASS	PASS
axis_euclid	Euclid	PASS	PASS	PASS	PASS	PASS	PASS	PASS
axi4_ribm	riBM	PASS	PASS	PASS	PASS	PASS	PASS	PASS
axi4_euclid	Euclid	PASS	PASS	PASS	PASS	PASS	PASS	PASS

8.2 Correction boundary

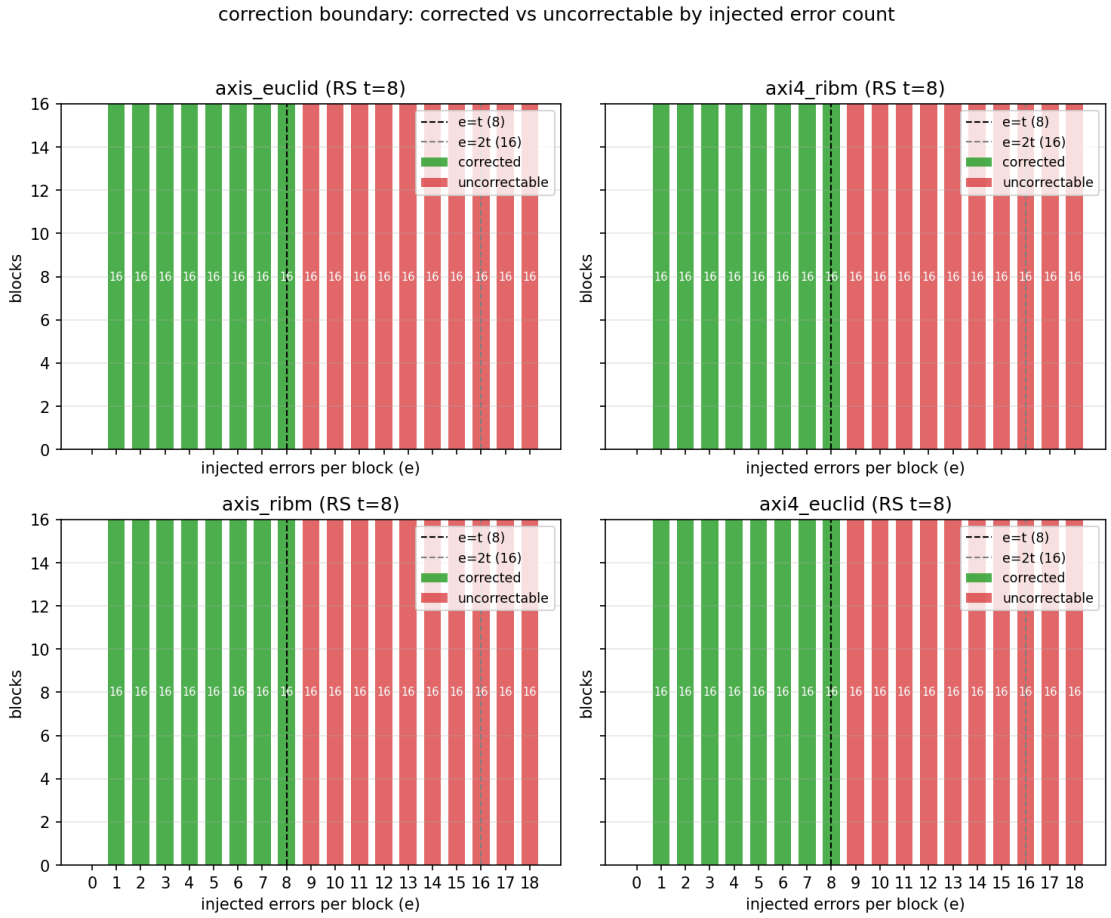


fig_1_correction_boundary.png

8.2.1 Figure 4.1: Reed-Solomon correction boundary across all four images.

8.3 Decode cost

Fig 2: decode cost

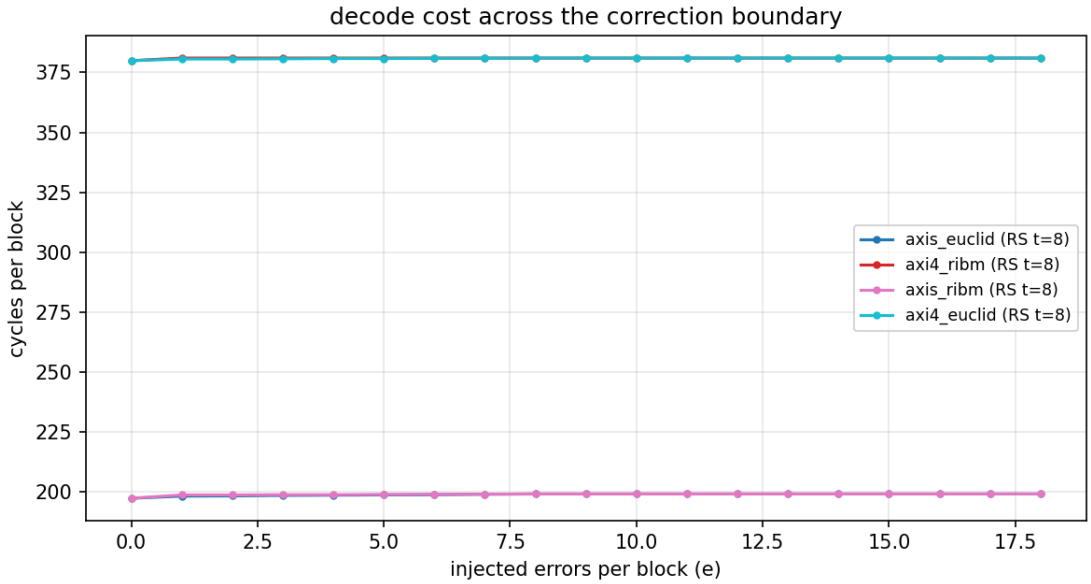


fig_2_decode_cost.png

8.3.1 Figure 4.2: Reed-Solomon decode cost across all four images.

8.4 Sweep verdict table

The sweep ran e = 0 .. 18 with 16 blocks per count on every image:

Table 4.2: Reed-Solomon sweep verdicts.

e	Verdict	Blocks	Notes
0	Correct	16	Clean decode.
1	Correct	16	16 symbols corrected.
2	Correct	16	32 symbols corrected.
3	Correct	16	48 symbols corrected.
4	Correct	16	64 symbols corrected.
5	Correct	16	80 symbols corrected.
6	Correct	16	96 symbols corrected.
7	Correct	16	112 symbols corrected.
8	Correct	16	128 symbols corrected.
9	Uncorrectable	16	Past correction boundary.
10	Uncorrectable	16	Past correction boundary.
11	Uncorrectable	16	Past correction boundary.
12	Uncorrectable	16	Past correction boundary.
13	Uncorrectable	16	Past correction boundary.
14	Uncorrectable	16	Past correction

e	Verdict	Blocks	Notes
			boundary.
15	Uncorrectable	16	Past correction boundary.
16	Uncorrectable	16	Past correction boundary.
17	Uncorrectable	16	Past correction boundary.
18	Uncorrectable	16	Past correction boundary.

8.5 Solver agreement

Fig 5: solver agreement

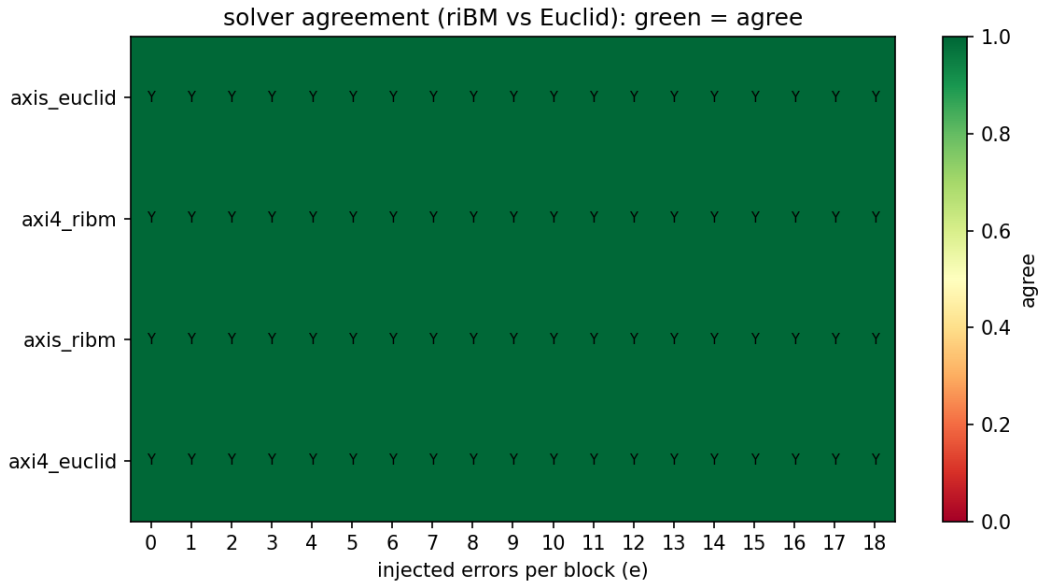


fig_5_solver_agreement.png

8.5.1 Figure 4.3: Reed-Solomon solver agreement matrix (riBM vs Euclid).

The matrix is entirely green with a Y in every cell. That means no image, at any injected error count from 0 to 18, disagreed with its sibling solver on the same deterministic workload. The absence of any red cell is the board evidence that the two key-equation solvers are consistent.

9 Soak Results

9.1 One million blocks on the AXI4 image

The 2026-10-05 soak ran the `axi4_ribm` image on board serial `200300B818A0`. Final counters from `stable/results/2026-10-05_soak/battery.json`:

Table 4.1: RS soak final counters.

Metric	Value
blocks_final	1,000,000
wall_s	34,902
blk_per_s	29.0
clean	113,143
corrected	768,077
uncorrectable	118,780
symbols_corrected	3,119,170
failing_runs	0
over_t_blocks	74,240
misdecoded	2
misdecode_rate	2.69e-05 (1 in 37,120)

The confidence statement is:

> 1,000,000 blocks, 3,119,170 symbols corrected, 0 failing runs, 9h42m at 29 blk/s UART-bound. Of the 74,240 blocks pushed past the correction limit, 2 were accepted and silently mis-decoded — a rate of 1 in 37,120, which is below the model's predicted roughly 1-in-20,000 mis-decode rate at e = t + 1 for this profile.

9.2 Mis-decodes versus the model

The soak's pass criterion is a bounded mis-decode rate, not zero mis-decodes. A block carrying more than t = 8 symbol errors is beyond the code's guaranteed radius; the decoder is mathematically permitted to return a codeword other than the transmitted one. The harness counts every such event, and the acceptance bound is the analytic failure rate for the profile. The observed 2 of 74,240 sits under the model bound, which is what "soak PASS" means here. Every other over-t block was flagged uncorrectable, and all 768,077 corrected blocks matched the oracle.

9.3 Soak timeline

Fig 3: soak timeline

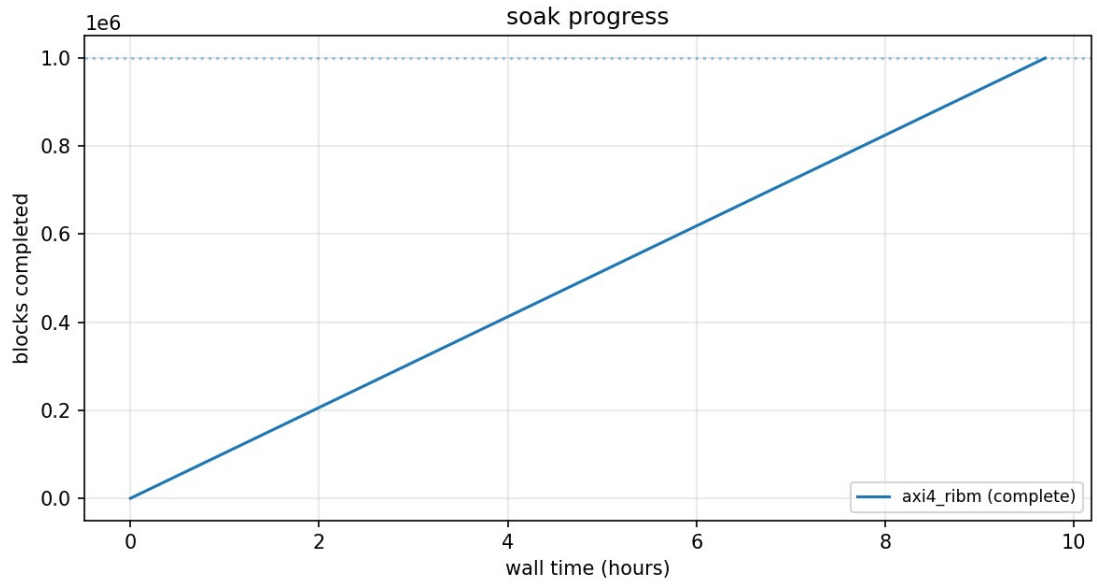


fig_3_soak_timeline.png

9.3.1 Figure 4.4: RS soak timeline over one million blocks.

9.4 What 29 blk/s means

The codec itself can run at roughly 500k blocks per second at 100 MHz. The soak measured 29 blocks per second because the host poll loop, running over a 115200-baud UART, is the bottleneck. That is not a hardware limit; it is a fixture limit. Any future soak that wants to go deeper or faster should size the host interaction accordingly, or move to a faster host interface, rather than inferring decoder throughput from the soak rate.

9.5 Image notes

The soak ran on the memory-to-memory AXI4 image. The earlier plan to prefer the AXIS flavour for soaks turned out to be unnecessary: the AXI4 job memories accept the soak's per-block job pattern without issue. The `axi4_ribm` image is single-decoder, so the riBM-versus-Euclid comparator counters read 0 by design; solver agreement on the board is covered by the deterministic cross-image campaigns in the battery chapter.

10 Honest Limits

10.1 What the board evidence does not cover

- The soak is statistical, not exhaustive. One million blocks is large, but it does not visit every possible error pattern. Two blocks out of 74,240 pushed past the correction limit were silently mis-decoded (1 in 37,120), below the model's predicted roughly 1-in-20,000 rate at e = t + 1; the soak's pass criterion is the bounded rate, not zero.
- The erasure `f = 2t` boundary is an open issue (TASK-005). f = t corrects and f = 2t+1 refuses on the board; f = 2t is flagged uncorrectable even though the design intent says it should correct.
- The dual-decoder comparator with `ENABLE_COMPARE=1` is sim-only on the board matrix. Solver agreement on the board is proven by running identical deterministic campaigns on the single-decoder images.
- BCH has no erasure path by design; that comparison is RS-only.
- The board validates only the RS(252,236) t=8, S=4 profile. Other profiles are covered in simulation and component DV.
- The component DV matrix exercises the codec in ways the board harness does not replicate.

10.2 Traceability

Every quantitative claim in this book can be traced to an artifact under `stable/`:

Table 5.1: Traceability from claim to artifact.

Claim	Artifact
Four images timing-clean at 100 MHz	`stable/reports/genesys2_{axis_ribm,axis_euclid,axi4_ribm,axi4_euclid}/timing_summary.txt`, all WNS positive, 0 failing endpoints.
7/7 campaigns PASS on all images	`stable/results/2026-10-05_battery/battery.json` and `FINDINGS.md`.
e <= 8 corrected, e > 8 uncorrectable	`stable/results/2026-10-05_battery/battery.json`, `sweep` array.
Solver agreement matrix all green	`stable/results/2026-10-05_battery/fig_5_solver_agreement.png`.
Million-block soak PASS on `axi4_ribm`	`stable/results/2026-10-05_soak/battery.json` and `FINDINGS.md`.
f = 2t erasure boundary open	`vault/tracking/projects/components/ecc-ip/reed-solomon/task/open/TASK-005.md`.